Inside UEFI: A Practical Guide to Protocol Development

Name : JaxLu

Date : 20250502

Ver : 0.0

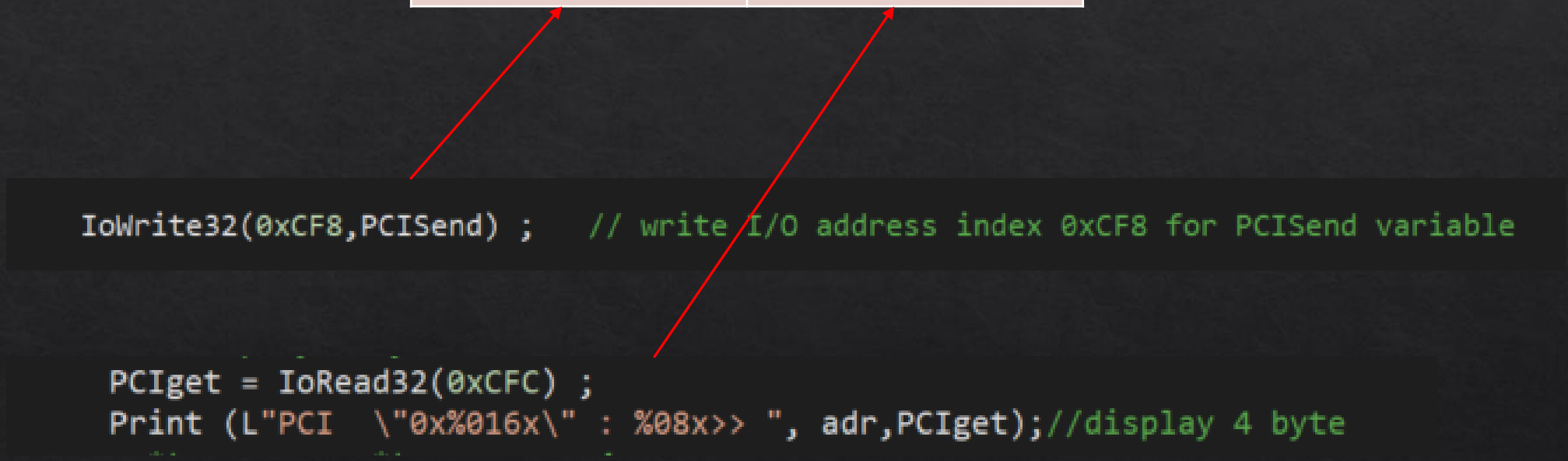
Agenda

- Use Protocol
- Create and Utilize the Protocol (Driver tutorial)

Use Protocol

Use Protocol

INDEX	DATA
CF8	CFC



```
IoWrite32(0xCF8,PCISend) ;    // write I/O address index 0xCF8 for PCISend variable
```

```
PCIget = IoRead32(0xCFC) ;
Print (L"PCI   \"0x%016x\" : %08x>> ", adr,PCIget);//display 4 byte
```

Use Protocol

14.2.8. EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Pci.Read()

14.2.9. EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Pci.Write()

Summary

Enables a PCI driver to access PCI controller registers in a PCI root bridge's configuration space.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_IO_MEM) (
  IN EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL               *This,
  IN EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_WIDTH         Width,
  IN UINT64                                        Address,
  IN UINTN                                         Count,
  IN OUT VOID                                      *Buffer
  );
```

Use Protocol

EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL

```
typedef struct _EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL {
  EFI_HANDLE
 ParentHandle;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_POLL_IO_MEM      PollMem;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_POLL_IO_MEM      PollIo;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_ACCESS           Mem;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_ACCESS           Io;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_ACCESS           Pci;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_COPY_MEM         CopyMem;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_MAP              Map;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_UNMAP            Unmap;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_ALLOCATE_BUFFER  AllocateBuffer;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_FREE_BUFFER      FreeBuffer;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_FLUSH            Flush;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_GET_ATTRIBUTES   GetAttributes;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_SET_ATTRIBUTES   SetAttributes;
  EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_CONFIGURATION    Configuration;
  UINT32                                           SegmentNumber;
}   EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL;
```

```
EFI_STATUS     Status = EFI_SUCCESS;
EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL     *PciRootBridgeIo = NULL; // PciRootBridgeIo
```

Use Protocol

```
EFI_STATUS     Status = EFI_SUCCESS;
EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL     *PciRootBridgeIo = NULL; // PciRootBridgeIo
```

LocateProtocol

```
typedef
EFI_STATUS
(EFIAPI *EFI_LOCATE_PROTOCOL) (
    IN EFI_GUID *Protocol,
    IN VOID *Registration, OPTIONAL
    OUT VOID **Interface
);
```

```
Status  = gBS->LocateProtocol (
                &gEfiPciRootBridgeIoProtocolGuid,
                 NULL,
                &PciRootBridgeIo
                );

if (EFI_ERROR (Status)) {
  Print (L"LocateProtocol gEfiPciRootBridgeIoProtocolGuid Error !!\n");
  return Status;
}
```

Protocol Guid,
NULL,
Interface protocol

Return value

返回值	描述
EFI_SUCCESS	A protocol instance matching Protocol was found and returned in Interface.
EFI_INVALID_PARAMETER	Interface is NULL.
EFI_NOT_FOUND	No protocol instances were found that match Protocol and Registration.

Note : LocateProtocol() Only find First support function and return point

Use Protocol

Pci.Read / Pci.Write:

14.2.8. EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Pci.Read()

14.2.9. EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Pci.Write()

Summary

Enables a PCI driver to access PCI controller registers in a PCI root bridge's configuration space.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_IO_MEM) (
  IN EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL              *This,
  IN EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_WIDTH        Width,
  IN UINT64                                       Address,
  IN UINTN                                        Count,
  IN OUT VOID                                     *Buffer
  );
```

```
Status  = PciRootBridgeIo->Pci.Read (
                      PciRootBridgeIo,
                      EfiPciWidthUint32,
                      adr,
                      1,
                     &PCIget
                      );
```

```
if (EFI_ERROR(Status)) {
  Print(L"Pci.Read error  Status: %r\n", Status);

}
else {
  Print(L"Pci.Read success PCIget: 0x%x\n", PCIget);

}
```

Status

EFI_SUCCESS	The data was read from or written to the PCI root bridge.
EFI_INVALID_PARAMETER	Width is invalid for this PCI root bridge.
EFI_INVALID_PARAMETER	Buffer is *NULL*.
EFI_OUT_OF_RESOURCES	The request could not be completed due to a lack of resources.

Create and Utilize the Protocol

Create and Utilize the Protocol

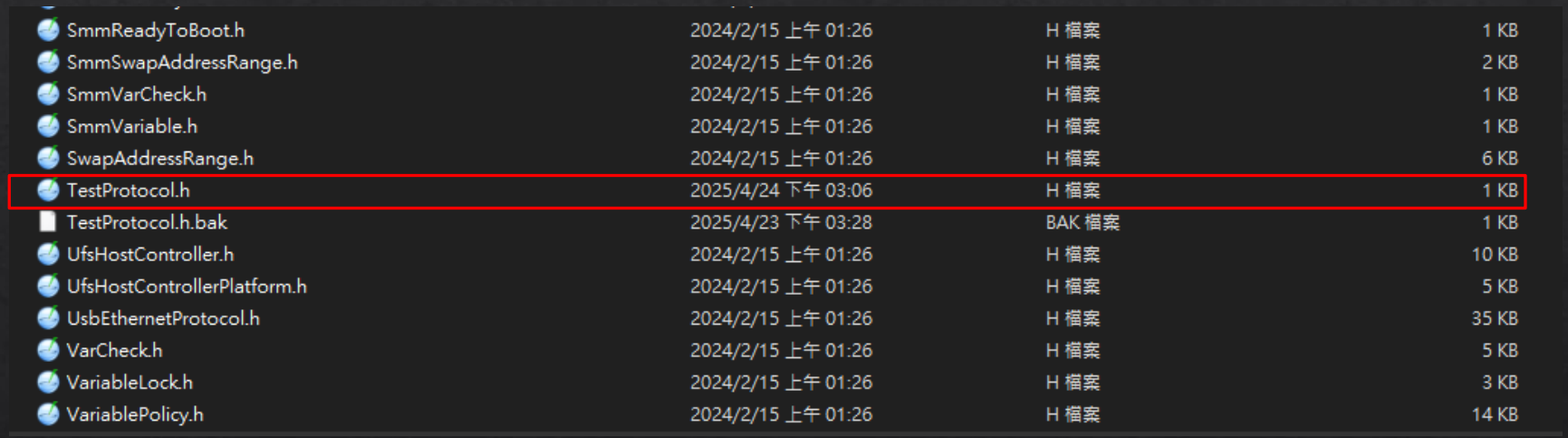
TestProtocol.h

```
//TestProtocol.h
#ifndef __HELLO_WORLD_PROTOCOL_H__
#define __HELLO_WORLD_PROTOCOL_H__

#include <Uefi.h>

#define EFI_HELLO_WORLD_PROTOCOL_GUID \
  {0x039d1af8, 0x1c8d, 0x408f, { 0x59, 0x25, 0x30, 0x4f, 0x5b, 0x96, 0x5d, 0x6e }}

typedef struct _EFI_HELLO_WORLD_PROTOCOL EFI_HELLO_WORLD_PROTOCOL;

typedef
EFI_STATUS
(EFIAPI *HELLOWORLD) (
  IN  EFI_HELLO_WORLD_PROTOCOL      *This  ,
  IN UINT64                         a,
  IN UINT64                         b,
  OUT UINT64                        *Result
  );

// new struct
typedef struct {
  HELLOWORLD Read;
  //HELLOWORLDWRITE Write;
} HELLO_WORLD_INTERFACE;


struct _EFI_HELLO_WORLD_PROTOCOL {
  UINTN                   Version;
  HELLO_WORLD_INTERFACE   HelloWorld;
};

extern EFI_GUID gEfiHelloWorldProtocolGuid;

#endif
```

Path: \MdeModulePkg\Include\Protocol

Create and Utilize the Protocol

TestProtocol.h



SmmReadyToBoot.h	2024/2/15 上午 01:26	H 檔案	1 KB
SmmSwapAddressRange.h	2024/2/15 上午 01:26	H 檔案	2 KB
SmmVarCheck.h	2024/2/15 上午 01:26	H 檔案	1 KB
SmmVariable.h	2024/2/15 上午 01:26	H 檔案	1 KB
SwapAddressRange.h	2024/2/15 上午 01:26	H 檔案	6 KB
TestProtocol.h	2025/4/24 下午 03:06	H 檔案	1 KB
TestProtocol.h.bak	2025/4/23 下午 03:28	BAK 檔案	1 KB
UfsHostController.h	2024/2/15 上午 01:26	H 檔案	10 KB
UfsHostControllerPlatform.h	2024/2/15 上午 01:26	H 檔案	5 KB
UsbEthernetProtocol.h	2024/2/15 上午 01:26	H 檔案	35 KB
VarCheck.h	2024/2/15 上午 01:26	H 檔案	5 KB
VariableLock.h	2024/2/15 上午 01:26	H 檔案	3 KB
VariablePolicy.h	2024/2/15 上午 01:26	H 檔案	14 KB

Path: \MdeModulePkg\Include\Protocol

Create and Utilize the Protocol

MdeModulePkg.dec

```
[Protocols]
  ## Load File protocol provides capability to load and unload EFI image into memory and execute it.
  #  Include/Protocol/LoadPe32Image.h
  #  This protocol is deprecated. Native EDKII module should NOT use this protocol to load/unload image.
  #  If developer need implement such functionality, they should use BasePeCoffLib.
```

```
  ## Include/Protocol/TestProtocol.h
  gEfiHelloWorldProtocolGuid = {0x039d1af8, 0x1c8d, 0x408f, { 0x59, 0x25, 0x30, 0x4f, 0x5b, 0x96, 0x5d, 0x6e }}
```

Path: \MdeModulePkg\MdeModulePkg.dec

Create and Utilize the Protocol

TestProtocolInstall.c

```c
// TestProtocolInstall.c
#include <Uefi.h>
#include <Library/UefiDriverEntryPoint.h>
#include <Library/UefiBootServicesTableLib.h>
#include <Library/MemoryAllocationLib.h>
#include <Library/DebugLib.h>
#include <Protocol/TestProtocol.h>
#include <Library/UefiLib.h>
#include <Library/UefiApplicationEntryPoint.h>

EFI_STATUS
EFIAPI
HelloWorld(
  IN  EFI_HELLO_WORLD_PROTOCOL      *This  ,
  IN UINT64                         a,
  IN UINT64                         b,
  OUT UINT64                        *Result
  )
{

  DEBUG ((EFI_D_ERROR, "Hello World_3\n"));
  *Result = (a + b) *2 ;
  return EFI_SUCCESS;
}

EFI_STATUS
EFIAPI
TestProtocolInstallEntry (
  IN  EFI_HANDLE                    ImageHandle,
  IN  EFI_SYSTEM_TABLE              *SystemTable
  )
{
  EFI_STATUS                  Status;
  EFI_HELLO_WORLD_PROTOCOL    *Protocol;

  Protocol = AllocatePool (sizeof (EFI_HELLO_WORLD_PROTOCOL));
  if (NULL == Protocol) {
    DEBUG ((EFI_D_ERROR, "[TestProtocol][%s][%d]:Out of resource.",__FUNCTION__, __LINE__));
    return EFI_OUT_OF_RESOURCES;
  }

  Protocol->Version = 0x6E;
//  Protocol->HelloWorld=HelloWorld;
  Protocol->HelloWorld.Read = HelloWorld;


  Status = gBS->InstallProtocolInterface (

                  );
  if (EFI_ERROR (Status)) {
    DEBUG ((EFI_D_ERROR, "[TestProtocol]Install EFI_HELLO_WORLD_PROTOCOL failed. - %r\n", Status));
    FreePool (Protocol);
    return Status;
  }

  return EFI_SUCCESS;
}
```

Create and Utilize the Protocol

TestProtocolInstall.c

```c
// TestProtocolInstall.c
#include <Uefi.h>
#include <Library/UefiDriverEntryPoint.h>
#include <Library/UefiBootServicesTableLib.h>
#include <Library/MemoryAllocationLib.h>
#include <Library/DebugLib.h>
#include <Protocol/TestProtocol.h>
#include <Library/UefiLib.h>
#include <Library/UefiApplicationEntryPoint.h>

EFI_STATUS
EFIAPI
HelloWorld(
  IN  EFI_HELLO_WORLD_PROTOCOL      *This  ,
  IN UINT64                         a,
  IN UINT64                         b,
  OUT UINT64                        *Result
  )
{

  DEBUG ((EFI_D_ERROR, "Hello World_3\n"));
  *Result = (a + b) *2 ;
  return EFI_SUCCESS;
}

EFI_STATUS
EFIAPI
TestProtocolInstallEntry (
  IN  EFI_HANDLE                    ImageHandle,
  IN  EFI_SYSTEM_TABLE              *SystemTable
  )
{
  EFI_STATUS                  Status;
  EFI_HELLO_WORLD_PROTOCOL    *Protocol;

  Protocol = AllocatePool (sizeof (EFI_HELLO_WORLD_PROTOCOL));
  if (NULL == Protocol) {
    DEBUG ((EFI_D_ERROR, "[TestProtocol][%s][%d]:Out of resource.",__FUNCTION__, __LINE__));
    return EFI_OUT_OF_RESOURCES;
  }

  Protocol->Version = 0x6E;
//  Protocol->HelloWorld=HelloWorld;
  Protocol->HelloWorld.Read = HelloWorld;


  Status = gBS->InstallProtocolInterface (

                  );
  if (EFI_ERROR (Status)) {
    DEBUG ((EFI_D_ERROR, "[TestProtocol]Install EFI_HELLO_WORLD_PROTOCOL failed. - %r\n", Status));
    FreePool (Protocol);
    return Status;
  }

  return EFI_SUCCESS;
}
```

Create and Utilize the Protocol

TestProtocolInstall.c

```c
// TestProtocolInstall.c
#include <Uefi.h>
#include <Library/UefiDriverEntryPoint.h>
#include <Library/UefiBootServicesTableLib.h>
#include <Library/MemoryAllocationLib.h>
#include <Library/DebugLib.h>
#include <Protocol/TestProtocol.h>
#include <Library/UefiLib.h>
#include <Library/UefiApplicationEntryPoint.h>

EFI_STATUS
EFIAPI
HelloWorld(
  IN  EFI_HELLO_WORLD_PROTOCOL      *This  ,
  IN UINT64                         a,
  IN UINT64                         b,
  OUT UINT64                        *Result
  )
{

  DEBUG ((EFI_D_ERROR, "Hello World_3\n"));
  *Result = (a + b) *2 ;
  return EFI_SUCCESS;
}

EFI_STATUS
EFIAPI
TestProtocolInstallEntry (
  IN  EFI_HANDLE                    ImageHandle,
  IN  EFI_SYSTEM_TABLE              *SystemTable
  )
{
  EFI_STATUS                  Status;
  EFI_HELLO_WORLD_PROTOCOL    *Protocol;

  Protocol = AllocatePool (sizeof (EFI_HELLO_WORLD_PROTOCOL));
  if (NULL == Protocol) {
    DEBUG ((EFI_D_ERROR, "[TestProtocol][%s][%d]:Out of resource.",__FUNCTION__, __LINE__));
    return EFI_OUT_OF_RESOURCES;
  }

  Protocol->Version = 0x6E;
//  Protocol->HelloWorld=HelloWorld;
  Protocol->HelloWorld.Read = HelloWorld;


  Status = gBS->InstallProtocolInterface (

                  );
  if (EFI_ERROR (Status)) {
    DEBUG ((EFI_D_ERROR, "[TestProtocol]Install EFI_HELLO_WORLD_PROTOCOL failed. - %r\n", Status));
    FreePool (Protocol);
    return Status;
  }

  return EFI_SUCCESS;
}
```

Create and Utilize the Protocol

TestProtocolInstall.c

```c
// TestProtocolInstall.c
#include <Uefi.h>
#include <Library/UefiDriverEntryPoint.h>
#include <Library/UefiBootServicesTableLib.h>
#include <Library/MemoryAllocationLib.h>
#include <Library/DebugLib.h>
#include <Protocol/TestProtocol.h>
#include <Library/UefiLib.h>
#include <Library/UefiApplicationEntryPoint.h>

EFI_STATUS
EFIAPI
HelloWorld(
  IN  EFI_HELLO_WORLD_PROTOCOL      *This  ,
  IN UINT64                         a,
  IN UINT64                         b,
  OUT UINT64                        *Result
  )
{

  DEBUG ((EFI_D_ERROR, "Hello World_3\n"));
  *Result = (a + b) *2 ;
  return EFI_SUCCESS;
}

EFI_STATUS
EFIAPI
TestProtocolInstallEntry (
  IN  EFI_HANDLE                    ImageHandle,
  IN  EFI_SYSTEM_TABLE              *SystemTable
  )
{
  EFI_STATUS                  Status;
  EFI_HELLO_WORLD_PROTOCOL    *Protocol;

  Protocol = AllocatePool (sizeof (EFI_HELLO_WORLD_PROTOCOL));
  if (NULL == Protocol) {
    DEBUG ((EFI_D_ERROR, "[TestProtocol][%s][%d]:Out of resource.",__FUNCTION__, __LINE__));
    return EFI_OUT_OF_RESOURCES;
  }

  Protocol->Version = 0x6E;
//  Protocol->HelloWorld=HelloWorld;
  Protocol->HelloWorld.Read = HelloWorld;


  Status = gBS->InstallProtocolInterface (

                  );
  if (EFI_ERROR (Status)) {
    DEBUG ((EFI_D_ERROR, "[TestProtocol]Install EFI_HELLO_WORLD_PROTOCOL failed. - %r\n", Status));
    FreePool (Protocol);
    return Status;
  }

  return EFI_SUCCESS;
}
```

Create and Utilize the Protocol

TestProtocolInstall.inf

```
[Defines]
  INF_VERSION                    = 0x00010015
  BASE_NAME                      = TestProtocolInstall
  FILE_GUID                      = 16BEFBED-60DC-4EA2-8E81-A343DF6C2117
  MODULE_TYPE                    = UEFI_DRIVER
  VERSION_STRING                 = 1.0
  ENTRY_POINT                    = TestProtocolInstallEntry

[Sources]
  TestProtocolInstall.c


[Packages]
        MdePkg/MdePkg.dec
        MdeModulePkg/MdeModulePkg.dec
        EmulatorPkg/EmulatorPkg.dec


[LibraryClasses]
        UefiDriverEntryPoint
        UefiBootServicesTableLib
        MemoryAllocationLib
        DebugLib

[Protocols]

   gEfiHelloWorldProtocolGuid

[Depex]
        TRUE
```

Create and Utilize the Protocol

TestProtocolUse.c

```c
#include <Uefi.h>
#include <Library/UefiDriverEntryPoint.h>
#include <Library/UefiBootServicesTableLib.h>
#include <Library/DebugLib.h>
#include <Protocol/TestProtocol.h>
#include <Library/UefiLib.h>
#include <Library/UefiApplicationEntryPoint.h>

EFI_STATUS
EFIAPI
TestProtocolUseEntry (
  IN  EFI_HANDLE                  ImageHandle,
  IN  EFI_SYSTEM_TABLE            *SystemTable
  )
{
  EFI_STATUS                  Status;
  EFI_HELLO_WORLD_PROTOCOL    *Protocol;
  UINT64  a = 1 ;
  UINT64  b = 2 ;
  UINT64  output = 0 ;

  Status = gBS->LocateProtocol (
            &gEfiHelloWorldProtocolGuid,
            NULL,
            (VOID **)&Protocol
        );  // Get protocol
  if (EFI_ERROR (Status)) {
    DEBUG ((EFI_D_ERROR, "[TestProtocol]Locate EFI_HELLO_WORLD_PROTOCOL failed. - %r\n", Status));
    return Status;
  }

  DEBUG ((EFI_D_ERROR, "Protocol Version: 0x%08x\n", Protocol->Version));

  Status = Protocol->HelloWorld.Read (
                               Protocol,
                               a,
                               b,
                               &output
                               );

  Print (L"Value %d !!\n",output);


  if (EFI_ERROR (Status)) {
    DEBUG ((EFI_D_ERROR, "[TestProtocol]Protocol->Hello failed. - %r\n", Status));
    return Status;
  }

  return EFI_SUCCESS;
}
```

Create and Utilize the Protocol

TestProtocolUse.c

LocateProtocol

HelloWorld.Read

```c
#include <Uefi.h>
#include <Library/UefiDriverEntryPoint.h>
#include <Library/UefiBootServicesTableLib.h>
#include <Library/DebugLib.h>
#include <Protocol/TestProtocol.h>
#include <Library/UefiLib.h>
#include <Library/UefiApplicationEntryPoint.h>

EFI_STATUS
EFIAPI
TestProtocolUseEntry (
  IN  EFI_HANDLE                  ImageHandle,
  IN  EFI_SYSTEM_TABLE            *SystemTable
  )
{
  EFI_STATUS                  Status;
  EFI_HELLO_WORLD_PROTOCOL    *Protocol;
  UINT64  a = 1 ;
  UINT64  b = 2 ;
  UINT64  output = 0 ;

  Status = gBS->LocateProtocol (
           &gEfiHelloWorldProtocolGuid,
           NULL,
           (VOID **)&Protocol
       );  // Get protocol
  if (EFI_ERROR (Status)) {
    DEBUG ((EFI_D_ERROR, "[TestProtocol]Locate EFI_HELLO_WORLD_PROTOCOL failed. - %r\n", Status));
    return Status;
  }

  DEBUG ((EFI_D_ERROR, "Protocol Version: 0x%08x\n", Protocol->Version));

  Status = Protocol->HelloWorld.Read (
                              Protocol,
                              a,
                              b,
                              &output
                              );

  Print (L"Value %d !!\n",output);

  if (EFI_ERROR (Status)) {
    DEBUG ((EFI_D_ERROR, "[TestProtocol]Protocol->Hello failed. - %r\n", Status));
    return Status;
  }

  return EFI_SUCCESS;
}
```

Create and Utilize the Protocol

TestProtocolUse.inf

```
[Defines]
  INF_VERSION                    = 0x00010015
  BASE_NAME                      = TestProtocolUse
  FILE_GUID                      = 16BAFDED-60DC-4EA2-8E81-A343DA6C2119
  MODULE_TYPE                    = UEFI_DRIVER
  VERSION_STRING                 = 1.0
  ENTRY_POINT                    = TestProtocolUseEntry

[Sources]
  TestProtocolUse.c


[Packages]
        MdePkg/MdePkg.dec
        MdeModulePkg/MdeModulePkg.dec
        EmulatorPkg/EmulatorPkg.dec


[LibraryClasses]
        UefiDriverEntryPoint
        UefiBootServicesTableLib
        MemoryAllocationLib
        DebugLib
        UefiLib|MdePkg/Library/UefiLib/UefiLib.inf

[Protocols]
         gEfiHelloWorldProtocolGuid

[Depex]
          TRUE
```

Create and Utilize the Protocol

MdeModulePkg.dsc

```
[Components]
  MdeModulePkg/Application/HelloWorld/HelloWorld.inf
  MdeModulePkg/Application/DumpDynPcd/DumpDynPcd.inf
  MdeModulePkg/Application/MemoryProfileInfo/MemoryProfileInfo.inf
  MdeModulePkg/Application/ExampleApp/ExampleApp.inf
  MdeModulePkg/Application/ExampleApp_2/ExampleAppEx2.inf

  MdeModulePkg/Application/Driver/TestProtocolInstall.inf
  MdeModulePkg/Application/Driver/TestProtocolUse.inf
```

Create and Utilize the Protocol

Result

```
fs0:\> cd EFI_Test

fs0:\EFI_Test> ls
Directory of: fs0:\EFI_Test

  02/25/25  12:28a <DIR>          32,768  .
  02/25/25  12:28a <DIR>               0  ..
  12/14/23  10:36p               425,280  RU.efi
  04/24/25  12:16p                20,032  keyboard_test.efi
  04/23/25  03:36p                10,208  TestProtocolUse.efi
  04/23/25  03:45p                 9,408  TestProtocolInstall.efi
          4 File(s)     464,928 bytes
          2 Dir(s)


fs0:\EFI_Test> load TestProtocolInstall.efi
load: Image fs0:\EFI_Test\TestProtocolInstall.efi loaded at 8B7E5000 - Success

fs0:\EFI_Test> load TestProtocolUse.efi
Protocol Version: 0x0000006E
Hello World_3
Value 6 !!
load: Image fs0:\EFI_Test\TestProtocolUse.efi loaded at 8B7E2000 - Success

fs0:\EFI_Test>
```

Reference

Driver :

UEFI——Protocol应用(简单的Driver)_uefi protocol-CSDN博客

UEFI-PCI :

https://uefi.org/specs/UEFI/2.10/14_Protocols_PCI_Bus_Support.html#id8

- UEFI protocol use

http://yiiyee.cn/blog/2019/10/02/uefi%E5%BC%80%E5%8F%91%E6%8E%A2%E7%B4%A242-protocol%E7%9A%84%E4%BD%BF%E7%94%A81/